

Détection de motifs ADN par Échantillonnage de Gibbs

Projet de Processus Stochastiques

Franck Duval HEUBA

Mai 2026

Le Problème : Découverte de motifs

- **Objectif** : Identifier une séquence courte et conservée (motif) cachée dans plusieurs séquences d'ADN.
- **Inconnues** :
 - 1 Le profil statistique du motif Θ (PWM).
 - 2 Les positions de départ $\mathcal{A} = \{A_1, \dots, A_N\}$ dans chaque séquence.
- **Difficulté** : L'espace des solutions est immense et parsemé d'optima locaux.
- **Solution** : Utiliser un algorithme MCMC (Gibbs) pour explorer la distribution jointe $p(\Theta, \mathcal{A} \mid \mathcal{S})$.

La Loi de Dirichlet (L'A Priori)

- Définie sur un simplexe (somme des probabilités = 1).
- Densité pour une colonne j : $p(\Theta_j|\alpha) \propto \prod_{i \in \{A,C,G,T\}} \Theta_{i,j}^{\alpha_i-1}$
- α_i : Pseudo-comptes (connaissance a priori).

L'Échantillonnage de Gibbs (MCMC)

- Simule une distribution jointe complexe par échantillonnages successifs selon les **lois conditionnelles complètes**.
- Garantit la convergence vers la distribution cible via la condition de **balance détaillée**.

Démonstration : Conjugaison Dirichlet-Multinomiale

- ❶ **Théorème de Bayes** : $p(\Theta \mid \mathcal{A}, \mathcal{S}) \propto p(\mathcal{S} \mid \mathcal{A}, \Theta) \times p(\Theta)$
- ❷ **Indépendance des colonnes** : On suppose que chaque position j du motif est statistiquement indépendante $p(\Theta) = \prod_{j=1}^W p(\Theta_j)$.
- ❸ **A Priori (Dirichlet)** : $p(\Theta_j) \propto \prod_i \Theta_{i,j}^{\alpha_i - 1}$
- ❹ **Vraisemblance (Multinomiale)** : Les comptages $n_{i,j}$ dans les fenêtres suivent une loi multinomiale : $p(\mathcal{S} \mid \mathcal{A}, \Theta) \propto \prod_j \prod_i \Theta_{i,j}^{n_{i,j}}$

Démonstration : Conjugaison Dirichlet-Multinomiale

- 5 **Fusion par produit** : On multiplie l'a priori et la vraisemblance.
- 6 **Simplification des exposants** ($x^a \cdot x^b = x^{a+b}$) :

$$p(\Theta_j | \mathcal{A}, \mathcal{S}) \propto \prod_{i \in \{A, C, G, T\}} \Theta_{i,j}^{(\alpha_i + n_{i,j}) - 1} \quad (1)$$

- 7 **Identification** : On reconnaît la forme d'une loi de Dirichlet.

Conclusion (Propriété de Conjugaison)

$$\Theta_j | \mathcal{A}, \mathcal{S} \sim \text{Dirichlet}(\alpha_A + n_{A,j}, \alpha_C + n_{C,j}, \alpha_G + n_{G,j}, \alpha_T + n_{T,j})$$

C'est cette mise à jour qui est implémentée dans le code Python.

Échantillonnage des positions : Fondements

Nous cherchons à échantillonner la position A_k du motif dans la séquence S_k , sachant le profil Θ et le modèle de fond Φ .

- **Objectif** : Calculer la distribution discrète $p(A_k = a \mid \Theta, S_k, \Phi)$.
- **Hypothèse** : On suppose un **a priori uniforme** sur les positions possibles ($p(A_k = a) = \text{constante}$).
- **Règle de Bayes** :

$$P(A_k = a \mid \Theta, S_k, \Phi) = \frac{P(S_k \mid A_k = a, \Theta, \Phi)}{\sum_{a'} P(S_k \mid A_k = a', \Theta, \Phi)} \quad (2)$$

Démonstration : Le ratio de vraisemblance

Pour simplifier le calcul, on divise la vraisemblance par la probabilité constante $P(S_k | \Phi)$ (probabilité que la séquence soit du "fond" pur).

- **Vraisemblance avec motif en a :**

$$P(S_k | A_k = a, \dots) = \underbrace{\prod_{j=0}^{W-1} \Theta_{S_{k,a+j},j}}_{\text{Fenêtre Motif}} \times \underbrace{\prod_{t \notin [a,a+W-1]} \Phi_{S_{k,t}}}_{\text{Reste (Fond)}}$$

- **Division par le Fond :** $P(S_k | \Phi) = \prod_t \Phi_{S_{k,t}}$
- **Résultat (Poids w_a) :** Les termes hors-fenêtre s'annulent.

$$w_a = \frac{P(S_k | A_k = a, \Theta, \Phi)}{P(S_k | \Phi)} = \prod_{j=0}^{W-1} \frac{\Theta_{S_{k,a+j},j}}{\Phi_{S_{k,a+j}}} \quad (3)$$

Une fois les poids w_a calculés pour chaque position de départ possible $a \in \{1, \dots, L - W + 1\}$, on obtient la distribution de probabilité :

Distribution discrète finale

$$P(A_k = a \mid \Theta, S_k, \Phi) = \frac{w_a}{\sum_{a'=1}^{L-W+1} w_{a'}} \quad (4)$$

- **Interprétation** : Plus le ratio Motif/Fond est élevé sur une fenêtre, plus la probabilité d'y échantillonner le motif est forte.
- **Implémentation** : Tirage selon cette loi multinomiale (méthode de la transformée inverse).

Implémentation : Boucle principale

```
1 def gibbs_motif_discovery(seq_file, W, phi, iterations=1000):
2     sequences = load_sequences(seq_file)
3     positions = [np.random.randint(0, len(s)-W+1) for s in sequences]
4
5     for i in range(iterations):
6         # Etape A : Profil (Theta)
7         counts = get_counts(sequences, positions, W)
8         theta = sample_theta(counts)
9         # Etape B : Positions (A_k)
10        positions = sample_positions(sequences, theta, phi, W)
11        # Evasion : Phase Shift
12        if i % 10 == 0:
13            positions = phase_shift_move(sequences, positions, theta, phi, W)
14    return positions, theta
```

- **Alternance** : L'algorithme bascule entre l'espace des paramètres (Θ) et l'espace des variables cachées ($\mathcal{A}$).

Focus : Échantillonnage de Θ

```
1 def sample_theta(counts, alpha=1.0):
2     W = counts.shape[1]
3     theta = np.zeros((4, W))
4     for j in range(W):
5         # Chaque colonne est independante
6         # Dirichlet(alpha + comptages)
7         theta[:, j] = np.random.dirichlet(counts[:, j] + alpha)
8     return theta
```

- **numpy.random.dirichlet** : Permet un tirage vectoriel efficace.
- **alpha** : Assure la positivité stricte du profil pour éviter les divisions par zéro lors du calcul des ratios.

Focus : Échantillonnage des positions

```
1 for a in range(possible_starts):
2     w = 1.0
3     for j in range(W):
4         nuc = seq[a + j]
5         # Ratio de vraisemblance local
6         w *= (theta[nuc_to_idx[nuc], j] / phi[nuc_to_idx[nuc]])
7     weights[a] = w
8
9 # Tirage selon la distribution normalisee
10 prob = weights / np.sum(weights)
11 choix = np.random.choice(possible_starts, p=prob)
```

- On traite chaque séquence indépendamment.
- Le vecteur `weights` contient le poids de chaque "départ" possible.

Focus : Mouvement de Phase Shift

```
1 def phase_shift_move(sequences, positions, theta, phi, W):
2     shift = np.random.choice([-1, 1])
3     new_pos = [p + shift for p in positions]
4
5     # Verification des limites
6     if not are_within_bounds(new_pos, sequences, W):
7         return positions
8
9     # Acceptation heuristique (Probabilite fixe de 30%)
10    if np.random.rand() < 0.3:
11        return new_pos
12    return positions
```

- **Metropolis-Hastings simplifié** : On accepte un décalage global pour s'aligner sur le "vrai" début du motif biologique.

Focus : Parallélisation (Restarts)

```
1 def run_gibbs_parallele(seq_file, W, phi, num_restarts=10):
2     with concurrent.futures.ProcessPoolExecutor() as executor:
3         taches = [(seq_file, W, phi, 500) for _ in range(num_restarts)]
4         resultats = executor.map(_executer_un_gibbs, taches)
5
6         for score, pos, theta in resultats:
7             if score > meilleur_score:
8                 meilleur_score, meilleures_pos = score, pos
9     return meilleures_pos, meilleur_theta
```

- **ProcessPoolExecutor** : Utilise tous les cœurs du CPU pour explorer l'espace de recherche en parallèle.
- **Critère de sélection** : Le score de consensus final.

Phase Shift :

- Propose un décalage global de ± 1 base.
- Acceptation stochastique (30%).
- Évite que le motif ne reste "décalé".

Redémarrages :

- 10 exécutions parallèles.
- Sélection par le **Score de Consensus**.
- $IC = \sum(2 - \text{Entropie})$.

Métrique 1 : Score de Consensus (IC)

Le score de consensus mesure la pureté statistique du motif trouvé à travers le **Contenu en Information** (Information Content).

- **Formule (pour une colonne j)** : $IC_j = 2 + \sum_{i \in \{A,C,G,T\}} p_{i,j} \log_2(p_{i,j})$
- **Représentation** : Il mesure la réduction d'incertitude (entropie). Un score de 2 bits signifie une conservation parfaite d'un seul nucléotide.
- **Pourquoi le calculer ?**
 - 1 C'est notre **fonction objectif** : l'échantillonneur de Gibbs cherche naturellement à maximiser cette conservation.
 - 2 Il sert de critère de sélection pour garder la meilleure chaîne lors des redémarrages multiples.

Métrique 2 : Average Jaccard Index (AJI)

L'AJI permet d'évaluer la précision spatiale de l'algorithme par rapport à une vérité terrain (Ground Truth).

- **Formule (Indice de Jaccard J) :**

$$J(V, P) = \frac{\text{Intersection}(V, P)}{\text{Union}(V, P)} = \frac{W - |\text{vrai_debut} - \text{pred_debut}|}{W + |\text{vrai_debut} - \text{pred_debut}|}$$

- **Représentation** : Un score de 1.0 indique un alignement parfait. Un score de 0.5 signifie que la moitié du motif est correctement localisée.
- **Pourquoi le calculer ?**
 - 1 Pour **valider l'algorithme** sur des données synthétiques où l'on connaît la réponse.
 - 2 Pour quantifier la difficulté de détection sur des données biologiques (PurR) où le signal est "dégénéré".

La Compétition : Optimisation de la largeur W

- **Problème** : Comment choisir la largeur optimale W quand elle est inconnue?
- **Approche** : Tester une plage de valeurs (ex : $W \in [15, 30]$).
- **Métrique de Sélection** : La **Densité d'Information**.

$$\text{Densité} = \frac{\text{Score de Consensus}}{W} \quad (5)$$

- **Pourquoi ?** Le score total augmente naturellement avec W . La densité permet de trouver le "cœur" du motif là où l'information est la plus concentrée par colonne.

Estimation du Fond

Le modèle Φ est estimé empiriquement sur l'ensemble des séquences de la compétition pour une précision maximale.

Jeu de données	AJI (Précision)	Consensus
Données Artificielles	0.8001	8.71 bits
Données PurR (Biologiques)	0.4430	13.73 bits

- **Analyse** : Succès sur données synthétiques (signal pur).
- **Limites Bio** : La dégénérescence des sites de fixation et les biais du génome (modèle de fond d'ordre 0 trop simple) expliquent la baisse d'AJI sur PurR.

- **Bibm@th.net** : Bibliothèque complète des mathématiques (cours, exercices, dictionnaire).
- **Khan Academy** : Plateforme de cours et exercices de mathématiques (collège au supérieur).
- **Jaicompris.com** : Cours et exercices corrigés en vidéo pour prépa et lycée.

Merci de votre attention !